Q1. Develop a Food Explorer Application using React and React Router. (50 Marks)

Use the following array of food items:

```
const foods = [
  { id: 1, name: "Pizza", category: "Italian", price: 250 },
  { id: 2, name: "Dosa", category: "South Indian", price: 100 },
  { id: 3, name: "Pasta", category: "Italian", price: 200 },
  { id: 4, name: "Sandwich", category: "Fast Food", price: 120 }
];
```

Instructions:

1. Configure the following routes using React Router:
    - `/` – Display a Home Page with a welcome message.
    - `/foods` – Display a Foods Page containing the food list and a search box.
2. Use `NavLink` to navigate between the Home Page and Foods Page. Highlight the active link.
3. Display a “Page Not Found” message for invalid URLs.
4. Create a functional component named `FoodCard` that accepts `name`, `category`, and `price` as props and displays the food details.
5. Use `.map()` to display all food items and assign a unique `key` to each component.
6. Display “Premium Item” in green if the price is ₹200 or above. Otherwise, display “Budget Friendly” in blue.
7. Add a search box using `useState` to filter food items by name, ignoring uppercase and lowercase differences. Display “No food items found” if no item matches the search text.

==========

Q2. Develop a Shopping List Manager using React and Redux Toolkit. (50 Marks)

Use the following array of shopping items:

```
const items = [
  { id: 1, name: "Milk" },
  { id: 2, name: "Bread" },
  { id: 3, name: "Apples" }
];
```

Instructions:

1. Create a functional component named `ShoppingItem` that accepts `name` as a prop and displays the item name.
2. Use `.map()` to display all shopping items and assign a unique `key` to each component.
3. Create a Redux Toolkit slice named `shoppingSlice` with an initial state of `{ items: items }`.
4. Define `addItem` and `removeItem` reducers to add a shopping item and remove a selected item, respectively.
5. Configure the Redux store using `configureStore` and wrap the application with `Provider`.
6. Add a text input and an **Add Item** button. Use `useDispatch` to add a new item to the shopping list.
7. Add a **Remove** button beside each item to remove it using Redux actions.
8. Use `useSelector` to display the shopping list. Display “Shopping list is empty” when no items are available.

==========

Q3. Develop a Course Explorer Application using React and React Router. (50 Marks)

Use the following array of courses:

```
const courses = [
  { id: 1, name: "React JS", duration: 30, level: "Advanced" },
  { id: 2, name: "HTML & CSS", duration: 15, level: "Beginner" },
  { id: 3, name: "Node JS", duration: 25, level: "Intermediate" },
  { id: 4, name: "JavaScript", duration: 20, level: "Beginner" }
];
```

Instructions:

1. Configure the following routes using React Router:
   - `/` – Display a Home Page with a welcome message.
   - `/courses` – Display a Courses Page containing the course list and a search box.
2. Use `NavLink` to navigate between the Home Page and Courses Page. Highlight the active link.
3. Display a “Page Not Found” message for invalid URLs.
4. Create a functional component named `CourseCard` that accepts `name`, `duration`, and `level` as props and displays the course details.
5. Use `.map()` to display all courses and assign a unique `key` to each component.
6. Display “Short Course” in green if the duration is 20 hours or less. Otherwise, display “Long Course” in blue.
7. Add a search box using `useState` to filter courses by name, ignoring uppercase and lowercase differences. Display “No courses found” if no course matches the search text.

==========

Q4. Develop a Student Contact Manager using React and Redux Toolkit. (50 Marks)

Use the following array of student contacts:

```
const contacts = [
  { id: 1, name: "Aarav", email: "aarav@gmail.com" },
  { id: 2, name: "Diya", email: "diya@gmail.com" },
  { id: 3, name: "Rohan", email: "rohan@gmail.com" }
];
```

Instructions:

1. Create a functional component named `ContactCard` that accepts `name` and `email` as props and displays the contact details.
2. Use `.map()` to display all contacts and assign a unique `key` to each component.
3. Create a Redux Toolkit slice named `contactSlice` with an initial state of `{ items: contacts }`.
4. Define `addContact` and `removeContact` reducers to add a new contact and remove an existing contact, respectively.
5. Configure the Redux store using `configureStore` and connect the application using `Provider`.
6. Add text inputs for the contact name and email, along with an **Add Contact** button. Use `useDispatch` to add a new contact.
7. Add a **Delete** button beside each contact to remove it using the appropriate Redux action.
8. Use `useSelector` to display the contacts from the Redux store. Display “No contacts available” when the list is empty.

==========

Q5. Develop a Product Explorer Application using React and React Router. (50 Marks)

Use the following array of products:

```
const products = [
  { id: 1, name: "Laptop", category: "Electronics", price: 50000 },
  { id: 2, name: "Notebook", category: "Stationery", price: 80 },
  { id: 3, name: "Headphones", category: "Electronics", price: 1500 },
```

```
  { id: 4, name: "Pen", category: "Stationery", price: 20 }
];
```

Instructions:

1. Configure the following routes using React Router:
    - `/` – Display a Home Page with a welcome message.
    - `/products` – Display a Products Page containing the product list and a search box.
2. Use `NavLink` to navigate between the Home Page and Products Page. Highlight the active link.
3. Display a “Page Not Found” message for invalid URLs.
4. Create a functional component named `ProductCard` that accepts `name`, `category`, and `price` as props and displays the product details.
5. Use `.map()` to display all products and assign a unique `key` to each component.
6. Display “High Value” in green if the price is ₹1,000 or above. Otherwise, display “Affordable” in blue.
7. Add a search box using `useState` to filter products by name, ignoring uppercase and lowercase differences. Display “No products found” if no product matches the search text.

==========

Q6. Develop an Assignment Manager using React and Redux Toolkit. (50 Marks)

Use the following array of assignments:

```
const assignments = [
  { id: 1, title: "Complete HTML Assignment" },
  { id: 2, title: "Submit React Practical" },
  { id: 3, title: "Prepare UML Diagrams" }
];
```

Instructions:

1. Create a functional component named `AssignmentCard` that accepts `title` as a prop and displays the assignment title.
2. Use `.map()` to display all assignments and assign a unique `key` to each component.
3. Create a Redux Toolkit slice named `assignmentSlice` with an initial state of `{ items: assignments }`.
4. Define `addAssignment` and `removeAssignment` reducers to add a new assignment and remove an existing assignment, respectively.
5. Configure the Redux store using `configureStore` and wrap the application with `Provider`.

6. Add a text input and an **Add Assignment** button. Use `useDispatch` to add a new assignment when the button is clicked.
7. Add a **Delete** button beside each assignment to remove it using Redux actions.
8. Use `useSelector` to display the assignments from the Redux store. Display “No assignments available” when the list is empty.

==========

Q7. Develop a Travel Explorer Application using React and React Router. (50 Marks)

Use the following array of destinations:

```
const destinations = [
  { id: 1, name: "Jaipur", country: "India", budget: 12000 },
  { id: 2, name: "Paris", country: "France", budget: 80000 },
  { id: 3, name: "Goa", country: "India", budget: 10000 },
  { id: 4, name: "Dubai", country: "UAE", budget: 50000 }
];
```

Instructions:

1. Configure the following routes using React Router:
   - `/` – Display a Home Page with a welcome message.
   - `/destinations` – Display a Destinations Page containing the destination list and a search box.
2. Use `NavLink` to navigate between the Home Page and Destinations Page. Highlight the active link.
3. Display a “Page Not Found” message for invalid URLs.
4. Create a functional component named `DestinationCard` that accepts `name`, `country`, and `budget` as props and displays the destination details.
5. Use `.map()` to display all destinations and assign a unique `key` to each component.
6. Display “Budget Friendly” in green if the budget is ₹15,000 or less. Otherwise, display “Premium Trip” in blue.
7. Add a search box using `useState` to filter destinations by name, ignoring uppercase and lowercase differences. Display “No destinations found” if no destination matches the search text.

==========

Q8. Develop a Student Event Manager using React and Redux Toolkit. (50 Marks)

Use the following array of events:

```
const events = [
  { id: 1, title: "Technical Seminar" },
  { id: 2, title: "Coding Competition" },
  { id: 3, title: "Project Presentation" }
];
```

Instructions:

1. Create a functional component named `EventCard` that accepts `title` as a prop and displays the event title.
2. Use `.map()` to display all events and assign a unique `key` to each component.
3. Create a Redux Toolkit slice named `eventSlice` with an initial state of `{ items: events }`.
4. Define `addEvent` and `removeEvent` reducers to add a new event and remove an existing event, respectively.
5. Configure the Redux store using `configureStore` and connect the application using `Provider`.
6. Add a text input and an **Add Event** button. Use `useDispatch` to add a new event to the list.
7. Add a **Delete** button beside each event to remove it using the appropriate Redux action.
8. Use `useSelector` to display the events from the Redux store. Display “No events available” when the list is empty.